

Angular et communication vers une API

.IPSSI'

I

Introduction à Angular



- Angular et ses atouts
- Environnement de travail
- Les composants

II

Les fondamentaux



- Templates & Data Binding
- Directives
- Observables (RxJS)
- Gestion des événements
- Structure d'un projet

III

Routage & gestion de l'état



- Navigation et routes
- Paramètres d'URL
- Services & état partagé
- Input / Output
- Gestion d'état avec RxJS

IV

Communication avec une API



- HttpClient, Fetch & Axios
- Requêtes CRUD (GET, POST, PUT, DELETE)
- Programmation asynchrone
- Optimisation des performances



**Première
application Angular**



Pokedex



OpenWeather

Application Météo

Kahoot!

Minis Quiz

Déroulé de la journée

1

Qu'est-ce qu'Angular ? Avantages et cas d'usage

2

Installation de l'environnement (Node.js, npm, Angular CLI)

3

Composants et approche orientée composants

4

Création d'une première application Angular simple

Qu'est-ce qu'Angular ?

Un framework front-end complet développé et maintenu par Google, écrit en TypeScript.

- Permet de construire des applications web dynamiques (SPA – Single Page Application)
- Structure le code autour de composants réutilisables
- Fournit tout un écosystème intégré : routage, formulaires, HTTP, tests

Application traditionnelle vs SPA

Site classique

Chaque navigation = rechargement complet de la page depuis le serveur

Angular (SPA)

Une seule page HTML : le contenu se met à jour dynamiquement via JavaScript/TypeScript, sans rechargement

→ **Expérience plus fluide, proche d'une app native**

HISTOIRE

Une histoire portée par Google

2010

AngularJS

Créé par Misko Hevery (Google) — 1ère génération, JavaScript

2016

Angular 2+

Réécriture complète en TypeScript, architecture par composants

2026

Angular 22

Version actuelle — signaux, zoneless par défaut

ILS UTILISENT ANGULAR



SAMSUNG



Pourquoi choisir Angular ?



Framework complet

Tout est inclus : routage, formulaires, HTTP client, tests — pas besoin de multiplier les librairies tierces.



TypeScript natif

Typage statique : moins de bugs, meilleure autocomplétion, code plus maintenable à grande échelle.



Architecture en composants

Code découpé en blocs réutilisables et testables indépendamment.



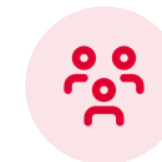
CLI puissante

Génération de code, build, tests et déploiement automatisés en une commande.



Performance

Détection de changements optimisée, lazy loading, compilation AOT.



Écosystème & communauté

Maintenu par Google, très utilisé en entreprise, documentation riche.

La dernière version

Angular 22

Sortie le 3 juin 2026 — cycle de release stable tous les 6 mois

- **Signal Forms**

Nouvelle API de formulaires basée sur les signaux, désormais stable

- **Zoneless par défaut**

Les nouveaux projets se passent de Zone.js pour la détection de changements

- **Composants "selectorless"**

Importer un composant directement dans un template, sans sélecteur

- **Angular Aria**

Primitives d'accessibilité prêtes à l'emploi, désormais stables

À retenir : pour cette formation, nous restons sur les fondamentaux (communs à toutes les versions récentes) — utilisables tels quels sur Angular 22.

Mettre en place l'environnement

Node.js

Environnement d'exécution JavaScript côté serveur, requis pour npm et Angular CLI (version LTS recommandée)

npm

Gestionnaire de paquets fourni avec Node.js — installe les librairies et dépendances du projet

Angular CLI

Outil en ligne de commande officiel pour créer, développer, tester et builder une app Angular

VS Code

Éditeur de code recommandé, avec extensions dédiées Angular (Angular Language Service)

Commandes essentielles

À exécuter dans un terminal

Vérifier l'installation de Node.js / npm

```
node -v  
npm -v
```

Installer Angular CLI globalement

```
npm install -g @angular/cli
```

Créer un nouveau projet Angular

```
ng new mon-projet
```

Lancer le serveur de développement

```
cd mon-projet  
ng serve --open
```

Commandes et liens utiles



Node.js

nodejs.org

```
node -v
```



npm

inclus avec Node.js

```
npm -v
```



Angular CLI

angular.dev/cli

```
npm i -g @angular/cli
```

Créer un projet

```
ng new mon-projet
```

Lancer le serveur

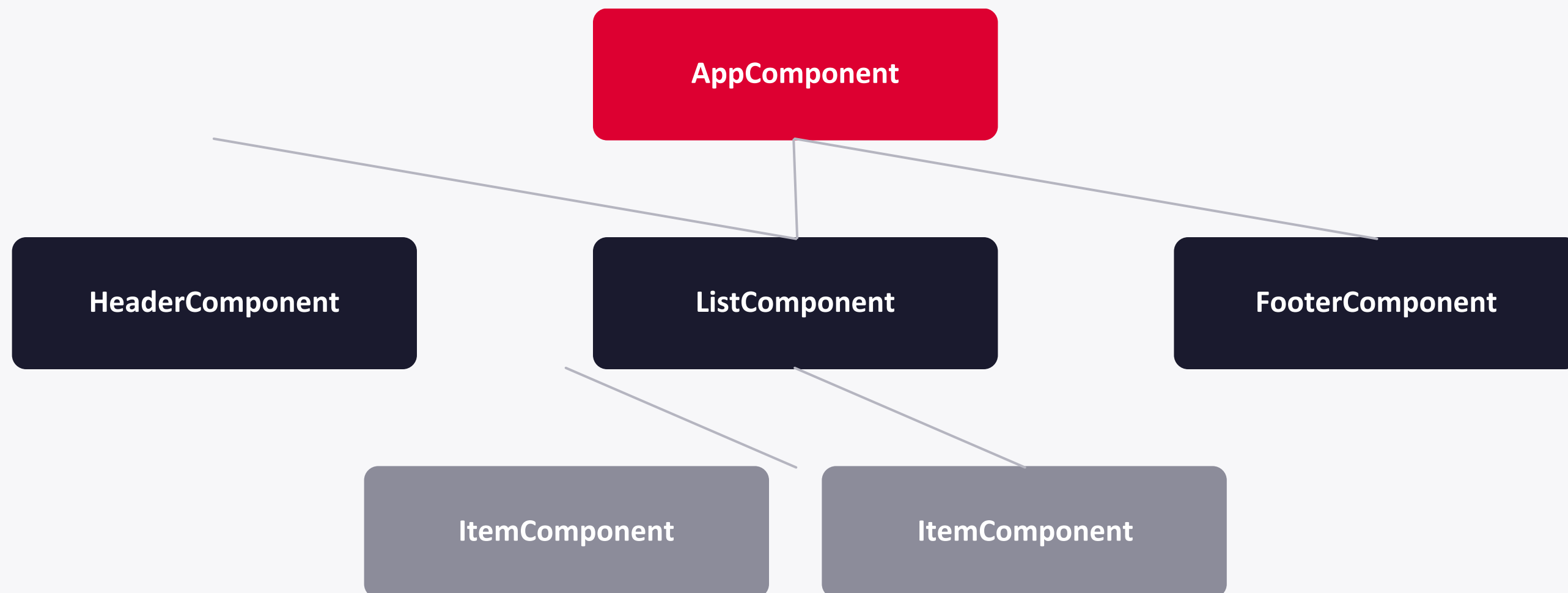
```
ng serve --open
```

Documentation officielle

```
https://angular.dev/overview
```

L'approche orientée composants

Une application Angular est un arbre de composants imbriqués, chacun responsable d'une portion d'interface autonome et réutilisable.



Chaque composant = un template (HTML) + une classe (TypeScript) + des styles (CSS)

Anatomie d'un composant

mon-composant.ts

Classe TypeScript : logique, données, décorateur
@Component

mon-composant.html

Template : structure HTML affichée à l'écran

mon-composant.css

Styles propres au composant (encapsulés)

mon-composant.spec.ts

Tests unitaires du composant <app-item></app-item>

```
@Component({
  selector: 'app-item',
  templateUrl:
    './item.component.html',
  styleUrls:
    ['./item.component.css']
})
export class ItemComponent {
  name: string = 'Angular';
}
```


Input, Output & cycle de vie

Communication parent ↔ enfant

Input reçoit une donnée du parent — **output()** émet un événement vers le parent.

```
export class ChildComponent {
  title= input.required<string>();

  selected = output <string>();

  onClick() {
    this.selected.emit(this.title);
  }
}
```

Utilisation côté parent :

```
<app-child [title]="name"
(selected)="onSelect($event)">
```

Cycle de vie d'un composant

Angular appelle ces méthodes automatiquement, dans l'ordre :

- 1 **ngOnChanges()** | **remplacé par effect()**
à chaque changement d'un @Input
- 2 **ngOnInit()**
une fois, juste après le 1er ngOnChanges
- 3 **ngAfterViewInit()**
une fois, après l'affichage de la vue
- 4 **ngOnDestroy()**
juste avant la destruction du composant

```
ngOnInit() {
  this.sub = this.api.getData()
    .subscribe(d => this.data = d);
}
ngOnDestroy() {
  this.sub.unsubscribe();
}
```

[Documentation & exemples : angular.dev/guide/components/lifecycle](https://angular.dev/guide/components/lifecycle)

Première application Angular

1

Générer le projet

```
ng new premiere-app
```

2

Se déplacer dans le dossier

```
cd premiere-app
```

3

Lancer le serveur local

```
ng serve --open
```

4

Modifier app.component.html

Éditer le template et observer le rechargement automatique

À faire en autonomie : personnaliser la page d'accueil avec votre nom et une courte présentation

Consignes — Première application Angular

Objectif : créer et personnaliser votre première application Angular (portfolio individuel).

À FAIRE

- 1 Générer le projet : `ng new mon-portfolio`
- 2 Lancer le serveur : `ng serve --open` et vérifier l'affichage
- 3 Modifier `app.component.html` : nom, photo, courte présentation
- 4 Personnaliser les styles (`app.component.css`)
- 5 Vérifier l'absence d'erreur dans la console du navigateur

RENDU ATTENDU

- Application fonctionnelle présentée au formateur en fin de séance
- Code déposé sur la plateforme indiquée par l'école avant la deadline communiquée

CRITÈRES D'ÉVALUATION

- Projet fonctionnel, sans erreur
- Personnalisation réelle du contenu
- Respect de la structure des composants

Ce qu'il faut retenir

- 1 Angular = framework front-end complet, en TypeScript, maintenu par Google
- 2 Une app Angular est structurée en composants réutilisables (HTML + TS + CSS)
- 3 Angular CLI simplifie la création, le développement et le build d'un projet
- 4 Prochaine étape (Jour 2) : templates, directives et gestion des événements

Déroulé de la journée

1

Templates & syntaxe de binding

2

Directives & composants

3

Services, observables & état

4

Événements, fichiers & scripts

Ce qu'on voit aujourd'hui



Templates Angular

Interpolation, bindings, nouvelle syntaxe @if / @for



Directives & composants

Directives structurelles, attributs, composants standalone



Services & injection

@Injectable, dependency injection, providedIn



Observables & état

RxJS, subscribe, gestion de l'état partagé



Événements utilisateur

(click), (input), \$event, formulaires simples



Organisation & scripts

Structure des dossiers, package.json, angular.json

Templates Angular : interpolation & bindings

Le template lie l'interface (HTML) à la classe du composant (TypeScript).

Interpolation

`{{ expression }}`

Affiche une valeur dans le HTML

```
<h1>Bonjour {{ userName }}</h1>
```

Property binding

`[ propriété ]`

Lie une propriété DOM à une valeur TS

```
<img [src]="photoUrl">
```

Event binding

`( événement )`

Réagit à une action de l'utilisateur

```
<button (click)="save()">Valider</button>
```

Two-way binding

`[( ngModel )]`

Combine property + event binding

```
<input [(ngModel)]="userName">
```

Templates Angular : flux de contrôle

Syntaxe Angular 22

@if / @for / @switch remplacent désormais *ngIf / *ngFor / *ngSwitch (dépréciées, suppression prévue en Angular 22).

```
@if (isLoggedIn) {  
  <div>Bienvenue, {{ userName }}!</div>  
} @else {  
  <p>Veuillez vous connecter</p>  
}
```

@if

```
@for (item of items; track item.id) {  
  <li>{{ item.name }}</li>  
} @empty {  
  <li>Aucun élément</li>  
}
```

@for

```
@switch (status) {  
  @case ('loading') { <p>Chargement...</p> }  
  @case ('error') { <p>Erreur !</p> }  
  @default { <p>OK</p> }  
}
```

[Documentation & exemples : angular.dev/guide/templates/control-flow](https://angular.dev/guide/templates/control-flow)

Les directives Angular

Une directive modifie l'apparence ou le comportement d'un élément du DOM. Il en existe 3 types :

Composant

Directive avec son propre template (vue en cascade — voir Jour 1)

`@Component({ ... })`

Structurelle

Modifie la structure du DOM en ajoutant/retirant des éléments

`@if, @for, @switch`

Attribut

Modifie l'apparence ou le comportement d'un élément existant

`NgClass, NgStyle`

Exemple — directives d'attribut NgClass et NgStyle

```
<div [ngClass]="{ actif: isActive, erreur: hasError }">
  Statut de la tâche
</div>

<p [ngStyle]="{ color: isUrgent ? 'red' : 'black' }">
  Priorité
</p>
```

Formulaires Angular : les deux approches

Deux façons structurées de construire un formulaire, avec des compromis différents.

Template-driven

ngModel

La logique vit dans le HTML : binding bidirectionnel via ngModel, structure gérée automatiquement par Angular. Nécessite FormsModule.

```
<input name="nom"
  [(ngModel)]="nom" required />
<button (click)="onSubmit(f)">Ajouter</button>

onSubmit(form: NgForm) {
  if (form.invalid) return;
}
```

[(ngModel)]

binding two-way

required

validateur déclaratif

Reactive Forms

formGroup

La structure est déclarée explicitement en TypeScript avec FormGroup/FormControl et des Validators. Nécessite ReactiveFormsModule.

```
form = new FormGroup({
  nom: new FormControl(
    '', Validators.required
  )
});

onSubmit() { if (this.form.invalid) return; }
```

FormControl

champ défini en TS

Validators.required

validateur programmatique

À retenir — formulaire simple et rapide : Template-driven. Formulaire complexe, testé, ou avec validations avancées : Reactive Forms.

Composants standalone & imports

Depuis Angular 19, les composants sont standalone par défaut : chacun déclare directement ses propres dépendances (autres composants, directives, pipes), sans passer par un NgModule.

```
@Component({
  selector: 'app-task-list',
  imports: [CommonModule, TaskItemComponent],
  templateUrl: './task-list.component.html',
})
export class TaskListComponent {
  tasks = [{ id: 1, name: 'Réviser Angular' }];
}
```

À retenir : le tableau imports remplace la déclaration dans un NgModule — chaque composant est autonome et ne charge que ce dont il a réellement besoin.

Services & injection de dépendances

Un service regroupe une logique réutilisable (accès aux données, calculs...) partagée entre plusieurs composants.

1. Déclarer le service

```
@Injectable({
  providedIn: 'root'
})
export class TaskService {
  getTasks() {
    return ['Réviser', 'Coder', 'Tester'];
  }
}
```

2. L'injecter dans un composant

```
export class TaskListComponent {
  private taskService = inject(TaskService);

  tasks = this.taskService.getTasks();
}
```

providedIn: 'root' rend le service accessible dans toute l'application, sans avoir à le déclarer dans un module.

[Documentation & exemples : angular.dev/guide/di](https://angular.dev/guide/di)

Observables & gestion de l'état (RxJS)

Un Observable représente un flux de données dans le temps. On s'y abonne avec `subscribe()` pour en récupérer les valeurs au fur et à mesure.

Service exposant un Observable

```
export class TaskService {
  private tasks$ = new BehaviorSubject<string[]>([]);

  getTasks() {
    return this.tasks$.asObservable();
  }

  addTask(task: string) {
    this.tasks$.next([...this.tasks$.value, task]);
  }
}
```

S'abonner dans un composant

```
export class TaskListComponent implements OnDestroy {
  tasks: string[] = [];
  private sub: Subscription;

  constructor(private taskService: TaskService) {
    this.sub = this.taskService.getTasks()
      .subscribe(tasks => this.tasks = tasks);
  }

  ngOnDestroy() { this.sub.unsubscribe(); }
}
```

Bonne pratique : toujours se désabonner dans `ngOnDestroy()` pour éviter les fuites mémoire (sauf avec le pipe `async`, qui le fait automatiquement).

Événements & interactions utilisateur

La syntaxe (événement)="méthode()" permet de réagir à toute action de l'utilisateur. L'objet \$event donne accès aux détails de l'événement déclenché.

(click)

Clic sur un élément

(input)

Saisie dans un champ

(keyup.enter)

Touche Entrée relâchée

(submit)

Soumission d'un formulaire

Exemple — formulaire simple avec saisie et validation

```
<input type="text" [(ngModel)]="taskName"
  (keyup.enter)="addTask()" placeholder="Nouvelle tâche">

<button (click)="addTask()">Ajouter</button>

// Dans la classe du composant :
addTask() {
  if (this.taskName.trim()) {
    this.tasks.push(this.taskName);
    this.taskName = '';
  }
}
```

Organisation des fichiers et dossiers

Une structure de projet claire facilite la maintenance à mesure que l'application grandit.

```
▸ src/  
  ▸ app/  
    ▸ components/  
      ▪ task-item/  
      ▪ task-list/  
    ▸ services/  
      ▪ task.service.ts  
    ▸ models/  
      ▪ task.model.ts  
      ▪ app.component.ts  
  ▸ assets/  
  ▸ environments/
```

components/

Un sous-dossier par composant réutilisable de l'interface

services/

La logique métier et l'accès aux données (API, état partagé)

models/

Les interfaces TypeScript décrivant la forme des données

environments/

Les variables de configuration (dev, prod, clés API...)

Scripts complémentaires pour le développement

Commandes utiles au quotidien, définies dans package.json et exécutables via Angular CLI ou npm.

```
ng serve --open
```

Lance le serveur de développement avec rechargement à chaud

```
ng build
```

Compile l'application pour la production (dossier dist/)

```
ng test
```

Exécute les tests unitaires (Karma/Jasmine ou Jest)

```
ng lint
```

Vérifie la qualité et le style du code (ESLint)

```
ng generate component nom
```

Génère un nouveau composant (raccourci : ng g c nom)

```
npm run start
```

Alias courant vers ng serve, défini dans package.json

Ce qu'il faut retenir

1 4 types de binding : interpolation, property, event, two-way — et @if/@for pour le flux de contrôle

2 Un service centralise la logique métier, injecté via inject() ou le constructeur

3 Les observables (RxJS) gèrent des flux de données ,toujours se désabonner dans ngOnDestroy()

4 Une structure de dossiers claire (components/services/models) facilite la maintenance

Déroulé de la journée

1

Angular Router & navigation

2

Services, Observables & état partagé

3

HttpClient & méthodes GET/POST/PUT/DELETE

4

Réponses asynchrones avec RxJS

Ce qu'on voit aujourd'hui



Angular Router

Configuration des routes et navigation entre pages



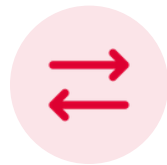
Paramètres d'URL

Passage et récupération de données via l'URL



État partagé

Services & Observables pour partager des données



Input / Output

Rappel : communication entre composants



HttpClient

Appels API : GET, POST, PUT, DELETE



Réponses asynchrones

Traitement et affichage des données avec RxJS

Angular Router : configurer les routes

Le Router associe une URL à un composant à afficher. Chaque route est définie dans un tableau.

1. Déclarer les routes (app.routes.ts)

```
export const routes: Routes = [  
  { path: '', component: HomeComponent },  
  { path: 'tasks',  
    component: TaskListComponent },  
  { path: 'tasks/:id',  
    component: TaskDetailComponent },  
  { path: '**', component: NotFoundComponent },  
];
```

2. Fournir le routeur (main.ts)

```
bootstrapApplication(AppComponent, {  
  providers: [  
    provideRouter(routes)  
  ]  
});
```

path: ''** capture toute route non trouvée (page 404). L'ordre des routes compte : la route générique doit toujours être déclarée en dernier.

[Documentation & exemples : angular.dev/guide/routing](https://angular.dev/guide/routing)

Navigation entre les pages

Deux façons de naviguer : déclarativement dans le template, ou depuis le code TypeScript.

Le point d'ancrage : `<router-outlet>`

```
<!-- app.component.html -->
<nav>
  <a routerLink="/">Accueil</a>
  <a routerLink="/tasks">Mes tâches</a>
</nav>

<router-outlet></router-outlet>
```

Navigation depuis le code (TypeScript)

```
export class TaskListComponent {
  private router = inject(Router);

  goToDetail(id: number) {
    this.router.navigate(['/tasks', id]);
  }
}
```

routerLinkActive ajoute automatiquement une classe CSS au lien correspondant à la page active : `<a routerLink="/tasks" routerLinkActive="active">`

Passage de paramètres & données d'URL

ActivatedRoute donne accès aux paramètres de la route active (ex : l'identifiant d'un élément).

Paramètre de route

`/tasks/:id`

`route.snapshot paramMap.get('id')`

Paramètre de requête

`/tasks?status=done`

`route.snapshot.queryParamMap.get('status')`

Exemple — récupérer l'identifiant dans le composant de détail

```
export class TaskDetailComponent implements OnInit {  
  private route = inject(ActivatedRoute);  
  taskId!: string;  
  
  ngOnInit() {  
    this.taskId = this.route.snapshot.paramMap.get('id')!;  
  }  
}
```

État partagé entre composants (Services & Observables)

Deux pages différentes (ex. : liste et détail) n'ont pas de lien direct — un service partagé, exposant un Observable, permet de synchroniser leur état.

```
@Injectable({ providedIn: 'root' })
export class TaskStateService {
  private selectedTask$ = new BehaviorSubject<Task | null>(null);
  readonly current$ = this.selectedTask$.asObservable();

  select(task: Task) {
    this.selectedTask$.next(task);
  }
}
```

État local (dans un composant) vs état partagé (dans un service) : local si la donnée ne concerne qu'un seul composant, partagé si plusieurs composants ou pages doivent y accéder ou la modifier.

Communication parent ↔ enfant

Rappel — Jour 1

Rappel express : dans une page liste + détail, @Input()/@Output() servent à faire dialoguer un composant parent et son composant enfant direct (à la différence du service, utile entre composants non liés).

```
// task-item.component.ts (enfant)
task = output<Task>();
selected = output<>();

onClick() { this.selected.emit(this.task); }
```

```
<!-- task-list.component.html (parent) -->
@for (t of tasks; track t.id) {
  <app-task-item
    [task]="t"
    (selected)="onTaskSelected($event)">
  </app-task-item>
}
```

[Détail complet & cycle de vie : voir Jour 1, slide 12](#)

RxJS : réagir aux événements et mettre à jour l'état

Les opérateurs RxJS transforment un flux d'événements (ex. : saisie utilisateur) en un flux de données prêt à afficher.

map

transforme chaque valeur émise

filter

ne garde que certaines valeurs

debounceTime

attend une pause avant d'émettre

switchMap

annule la requête précédente et bascule vers une nouvelle

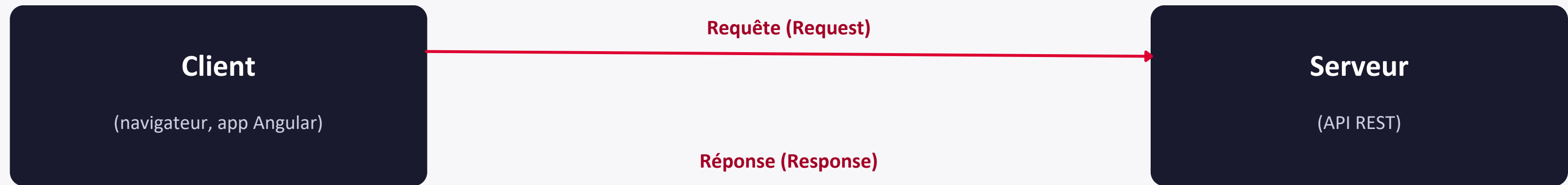
Exemple — recherche en direct avec debounce

```
this.searchInput$.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  switchMap(term => this.api.search(term))  
)  
.subscribe(results => this.results = results);
```

Chaque nouvelle saisie annule la requête précédente encore en cours : on n'affiche que le dernier résultat pertinent.

Le protocole HTTP : les bases

HTTP (HyperText Transfer Protocol) est le protocole de communication du web. Avant de coder avec HttpClient, il faut comprendre ce qui se passe « sous le capot ».



Sans état (stateless)

Chaque requête est indépendante : le serveur ne mémorise rien entre deux requêtes

Basé sur le texte

Requêtes et réponses sont lisibles, structurées en lignes de texte

Client-serveur

Le client initie toujours l'échange ; le serveur répond

[Documentation : developer.mozilla.org/fr/docs/Web/HTTP](https://developer.mozilla.org/fr/docs/Web/HTTP)

Anatomie d'une requête HTTP

Une requête envoyée par le client contient toujours ces éléments.

```
POST /api/tasks HTTP/1.1
Host: api.exemple.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

```
{
  "name": "Réviser Angular",
  "done": false
}
```

Ligne de requête

Méthode + chemin + version HTTP

```
POST /api/tasks HTTP/1.1
```

En-têtes (headers)

Métadonnées : format des données, authentification...

```
Content-Type, Authorization
```

Corps (body)

Les données envoyées (souvent en JSON), absent pour un GET

```
{ "name": "..." }
```


Anatomie d'une réponse HTTP

Le serveur répond avec une structure symétrique à celle de la requête.

```
HTTP/1.1 201 Created
Content-Type: application/json
Date: Tue, 11 Aug 2026 10:15:00 GMT

{
  "id": 42,
  "name": "Réviser Angular",
  "done": false
}
```

Ligne de statut

Version HTTP + code de statut + libellé

```
HTTP/1.1 201 Created
```

En-têtes (headers)

Format de la réponse, date, mise en cache...

```
Content-Type: application/json
```

Corps (body)

Les données renvoyées par le serveur, en JSON généralement

```
{ "id": 42, ... }
```

Les codes de statut HTTP

Le premier chiffre du code indique la catégorie de la réponse.

2xx

Succès

3xx

Redirection

4xx

Erreur client

5xx

Erreur serveur

200 OK

Requête réussie

201 Created

Ressource créée (POST)

204 No Content

Succès sans contenu (DELETE)

301 Moved Permanently

Ressource déplacée

304 Not Modified

Utiliser la version en cache

400 Bad Request

Requête mal formée

401 Unauthorized

Authentification requise

403 Forbidden

Accès refusé

404 Not Found

Ressource introuvable

500 Internal Server Error

Erreur inattendue côté serveur

503 Service Unavailable

Serveur temporairement indisponible

Astuce : dans Angular, on distingue une erreur (statut 4xx/5xx) d'un succès grâce au callback error de subscribe(), sans avoir à tester le code manuellement.

Qu'est-ce qu'une API REST ?

REST (Representational State Transfer) est un style d'architecture pour concevoir des API web, basé sur HTTP.

Ressources identifiées par une URL

`/tasks`, `/tasks/42`, `/users/7/tasks` — chaque ressource a son adresse unique

Verbes HTTP standards

GET, POST, PUT, DELETE expriment l'action, pas l'URL (pas de `/getTasks`)

Sans état (stateless)

Chaque requête contient tout le nécessaire (ex. : le token d'authentification)

Format d'échange standard

Les données circulent en JSON, un format texte lisible par l'humain et la machine

Exemple d'URLs REST bien conçues :

GET	<code>/tasks</code>	→ liste toutes les tâches
GET	<code>/tasks/42</code>	→ récupère la tâche n°42
POST	<code>/tasks</code>	→ crée une nouvelle tâche
PUT	<code>/tasks/42</code>	→ met à jour la tâche n°42
DELETE	<code>/tasks/42</code>	→ supprime la tâche n°42

Les en-têtes (headers) HTTP courants

Les headers transportent des métadonnées, séparées des données elles-mêmes (le body).

Content-Type

```
application/json
```

Indique le format des données envoyées ou reçues

Authorization

```
Bearer eyJhbGc...
```

Transporte le jeton d'authentification (JWT, API key...)

Accept

```
application/json
```

Indique au serveur le format de réponse souhaité

Cache-Control

```
no-cache
```

Contrôle la mise en cache de la réponse par le navigateur

En Angular, on ajoute des headers via HttpHeaders ou un intercepteur HTTP (pour l'authentification, par exemple).

HttpClient : appeler une API

HttpClient est le module Angular officiel pour communiquer avec une API REST. Chaque requête retourne un Observable.

1. Activer HttpClient (main.ts)

```
bootstrapApplication(AppComponent, {  
  providers: [  
    provideHttpClient()  
  ]  
});
```

2. Effectuer une requête GET

```
@Injectable({ providedIn: 'root' })  
export class TaskApiService {  
  private http = inject(HttpClient);  
  private url = 'https://api.exemple.com/tasks';  
  
  getTasks(): Observable<Task[]> {  
    return this.http.get<Task[]>(this.url);  
  }  
}
```

[Documentation & exemples : angular.dev/guide/http](https://angular.dev/guide/http)

GET, POST, PUT, DELETE

Chaque méthode HTTP correspond à une action CRUD (Create, Read, Update, Delete).

GET

Lire

```
getTasks() {  
  return this.http.get<Task[]>(this.url);  
}
```

POST

Créer

```
addTask(task: Task) {  
  return this.http.post<Task>(this.url, task);  
}
```

PUT

Modifier

```
updateTask(id: number, task: Task) {  
  return this.http.put<Task>(`${this.url}/${id}`, task);  
}
```

DELETE

Supprimer

```
deleteTask(id: number) {  
  return this.http.delete(`${this.url}/${id}`);  
}
```

Réponses asynchrones & affichage des données

Une requête HTTP est asynchrone : la réponse arrive plus tard. Deux approches pour l'afficher proprement.

Option 1 — subscribe() manuel

```
export class TaskListComponent implements OnInit {
  tasks: Task[] = [];
  loading = true;

  ngOnInit() {
    this.api.getTasks().subscribe({
      next: (data) => {
        this.tasks = data;
        this.loading = false;
      },
      error: (err) => console.error(err)
    });
  }
}
```

Option 2 — pipe async (recommandé)

```
export class TaskListComponent {
  tasks$ = this.api.getTasks();
}

<!-- template -->
@for (task of tasks$ | async; track task.id) {
  <li>{{ task.name }}</li>
}
```

Le pipe async s'abonne et se désabonne automatiquement — plus besoin de ngOnDestroy() pour cet Observable.

Ce qu'il faut retenir

- 1 HTTP fonctionne en requête/réponse ; le code de statut (2xx/4xx/5xx) indique le résultat
- 2 Une API REST expose des ressources via des URLs, manipulées avec les verbes HTTP standards
- 3 Le Router associe une URL à un composant ; ActivatedRoute donne accès aux paramètres
- 4 Un service partagé (avec Observable) synchronise l'état entre plusieurs composants ou pages
- 5 HttpClient couvre les 4 méthodes CRUD : GET, POST, PUT, DELETE — chaque appel retourne un Observable